- "How would you handle situations where the user denies camera access on a sensitive page, such as an identity verification flow?"
- "Describe how you would implement camera selection for devices with multiple cameras, and how would you test this across different browsers and devices?"
- "How do you ensure accessibility when implementing camera features in a web application?"
- "How do you ensure the security and privacy of camera data, especially when integrating with third-party APIs or cloud services for image processing?"
- "What strategies would you use to minimize the impact on device resources when using the camera in a web application, especially on mobile devices?"

Initial question: "How do you implement clipboard functionality in a web application?"

Analysis: This question tests a candidate's ability to integrate clipboard features securely and compatibly. Candidates should show knowledge of secure contexts, browser support, permissions, and user-triggered actions. The best answers address error handling, fallbacks for unsupported browsers, and clear user communication.

Answer: Clipboard functionality in a web application is typically implemented using the Clipboard API, which provides asynchronous methods for reading from and writing to the system clipboard via the `navigator.clipboard` object. To copy text, use `navigator.clipboard.writeText()`, and to read text, use `navigator.clipboard.readText()`. Both methods return Promises, making them easy to integrate with modern JavaScript code.

When implementing clipboard features, several considerations are important. Clipboard access is only available in secure contexts (such as HTTPS, localhost, or file URLs), and browser support should be checked to ensure compatibility across different environments. Permissions and security are critical: browsers may require explicit user permission, especially for reading clipboard data, and may prompt the user or restrict access if permissions are not granted.